f591c58d-5194-49f7-aff6-235cfb61d4ec

<prev_user>

<request>

You are working on a task in the 'zhaohongxuan/obsidian-weread-plugin' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/obsidian-weread-plugin/obsidian-weread-plugin (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-02T05:07:59.633Z</current_datetime>

Consider the following problem statement:

<problem_statement>
我想做一个微信读书的gallary或者说书架 的command，展示本地vault和远程书架上的数据，比如哪些是在本地vault的，但是不满足当前设置中的过滤条件，比如公众号类型、或者划线条数，需要一个特殊标记，在书架上就展示封面、作者、进度、是否阅读完成（根据进度接口的完成时间）等关键信息，然后方便用户进行筛选。书架也需要进行懒加载，因为获取读书进度是一本本书进行的查询的，全部查询性能可能有问题。在设置页面也可以增加一个书架的入口。从gallary中应该很方便的一键点击进入本地文件。
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prev_assistant>

<plan>
这是一个需要代码改动的功能，但从你的描述看，更适合先按“功能设计/实施方案”来推进。

1. 先抽出“书架数据层”
- 在现有 `ApiManager`、`SyncNotebooks`、`FileManager` 之外新增一个专门的书架数据组装层。
- 统一合并两类数据：远程书架元数据（`/api/user/notebook`）和本地 vault 中已同步文件（frontmatter + 文件路径）。
- 为每本书生成书架卡片模型，至少包含：封面、书名、作者、bookId、划线数、评论数、本地文件路径、是否存在本地文件、远程最近阅读日期、进度加载状态、完成时间、阅读进度、是否公众号、是否命中过滤规则。

2. 明确“过滤状态”而不是只做简单布尔值
- 现有过滤逻辑分散在 `syncNotebooks.ts` 的 `filterNoteMetas` 中，书架功能需要把“为什么不会被同步”显式展示出来。
- 建议把过滤结果拆成多个原因维度，例如：
  - `excludedByArticleType`
  - `excludedByNoteCount`
  - `excludedByBlacklist`
  - `excludedByWhitelist`
  - `includedByCurrentSettings`
- 这样书架上就可以对“本地存在，但按当前设置不会参与“打开书架”的入口；命令面板和 ribbon 菜单也新增“打开微信读书书架”命令。
- 这样书架可以承载筛选、搜索、懒加载、打开本地文件等交互，而不会让设置页过重。

6. 书架界面建议支持的最小交互
- 搜索：按书名、作者过滤。
- 过滤器：全部 / 仅本地 / 仅远程 / 本地但不满足当前同步条件 / 已读完 / 未读完 / 公众号。
- 排序：最近阅读、进度、标题。
- 卡片操作：
  - 打开本地文件
  - 若本地不存在，可触发单本同步或提示需先同步
  - 显示过滤原因标签
- 卡片文案重点：
  - 封面
  - 书名 / 作者
  - 划线数
  - 进度百分比
  - 完成时间或最近阅读时间
  - 本地状态
  - 特殊标记（例如“公众号”“低于划线阈值”“黑名单”“未入白名单”）

7. 设置页入口的改法
- 在 `src/settingTab.ts` 增加一个“书架/画廊”设置区块。
- 这里不承担完整书架渲染，只提供：
  - 打开书架按钮
  - 可选的书架默认筛选配置（如果你后面想做）
- 这样满足“设置页面也可以增加入口”，同时保持设置页结构清晰。

8. 命令与入口改动点
- `main.ts` 里新增 command。
- ribbon 右键菜单里也建议新增“打开书架”入口，和现有“同步”“打开微信读书视图”保持一致。
- 如果采用独立 view，需要注册新的 view type，并封装激活逻辑，类似当前 `activateReadingView()`。

9. 打开本地文件能力
- 书架卡片应直接持有 `TFile` 或可解析到 `TFile` 的路径信息。
- 点击卡片主区域可打开详情/聚焦；点击“打开本地文件”按钮则直接在当前叶子页或新标签页中打开对应笔记。
- 对“本地存在但不满足当前同步条件”的书，这个能力尤其重要，因为它正是你这个需求的核心使用场景。

10. 与现有同步逻辑的边界
- 书架本身不应改写同步结果，也不要直接编辑用户文件。
- 它只是可视化当前“远程状态 + 本地状态 + 当前设置下的可同步性”。
- 过滤判断应尽量复用 `syncNotebooks.ts` 的规则来源，避免出现“书架显示能同步，但实际同步时被跳过”的分叉。

11. 推荐的实施顺序
- 先做数据模型与本地/远程合并。
- 再抽取“同步过滤原因”的判定函数。
- 然后做独立书架 view 的基础卡片列表。
- 再接入懒加载进度。
- 最后补设置入口、命令入口、打开本地文件、筛选与样式优化。

12. 这个方案在当前仓库里是可落地的
- 命令入口已有模式：`main.ts`
- 设置入口已有模式：`src/settingTab.ts`
- 本地文件索引已有基础：`src/fileManager.ts`
- 远程元数据已有基础：`parseMetadata()` + `ApiManager.getNotebooksWithRetry()`
- 单本进度接口已有基础：`ApiManager.getProgress()`
- 现有书籍卡片样式与选择器 UI 也能部分复用：`settingTab.ts` + `style.css`

如果你愿意，我下一步可以直接基于这个方案，给你拆成“最小可实现版本”的具体改动清单。
</plan>

<current>

<request>

You are working on a task in the 'zhaohongxuan/obsidian-weread-plugin' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/obsidian-weread-plugin/obsidian-weread-plugin (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-02T05:12:12.045Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Implement the plan:

这是一个需要代码改动的功能，但从你的描述看，更适合先按“功能设计/实施方案”来推进。

1. 先抽出“书架数据层”
- 在现有 `ApiManager`、`SyncNotebooks`、`FileManager` 之外新增一个专门的书架数据组装层。
- 统一合并两类数据：远程书架元数据（`/api/user/notebook`）和本地 vault 中已同步文件（frontmatter + 文件路径）。
- 为每本书生成书架卡片模型，至少包含：封面、书名、作者、bookId、划线数、评论数、本地文件路径、是否存在本地文件、远程最近阅读日期、进度加载状态、完成时间、阅读进度、是否公众号、是否命中过滤规则。

2. 明确“过滤状态”而不是只做简单布尔值
- 现有过滤逻辑分散在 `syncNotebooks.ts` 的 `filterNoteMetas` 中，书架功能需要把“为什么不会被同步”显式展示出来。
- 建议把过滤结果拆成多个原因维度，例如：
  - `excludedByArticleType`
  - `excludedByNoteCount`
  - `excludedByBlacklist`
  - `excludedByWhitelist`
  - `includedByCurrentSettings`
- 这样书架上就可以对“本地存在，但按当前设置不会参与同步”的书打特殊标记，而不是只显示“已过滤”。

3. 复用现有本地文件识别能力
- `fileManager.ts` 已经能扫描 vault 内所有微信读书笔记文件并识别

`bookId`。
- 书架应复用这套 frontmatter 识别逻辑，而不是重新扫描 Markdown 内容。
- 需要补一个“bookId -> 本地文件”索引，支持卡片上一键打开本地文件，并能区分：
  - 远程存在 + 本地存在
  - 仅远程存在
  - 仅本地存在（孤儿文件 / 当前远程已无此书）

4. 懒加载阅读进度
- 你提到的性能问题成立：`getProgress(bookId)` 目前是逐本请求，不能在书架首次打开时全量拉取。
- 书架列表首次只展示基础元数据；进入视口、翻页或用户点击“加载更多”时，再批量/分片加载当前页书籍的进度。
- 需要引入分页或游标式状态，至少支持：
  - 首屏卡片渲染
  - 当前页进度异步回填
  - 已加载结果缓存
  - 失败重试
- 如果用户切换筛选条件，只复用已有缓存，不重复请求已拿到的进度。

5. UI 形态建议用独立书架视图，而不是继续堆在设置页里
- 现有 `WereadReadingView` 说明插件已经有自定义 view 的模式。
- 书架更适合做成独立 `ItemView` 或专用 `Modal`，而不是把大量卡片塞进设置页。
- 设置页只保留“打开书架”的入口；命令面板和 ribbon 菜单也新增“打开微信读书书架”命令。
- 这样书架可以承载筛选、搜索、懒加载、打开本地文件等交互，而不会让设置页过重。

6. 书架界面建议支持的最小交互
- 搜索：按书名、作者过滤。
- 过滤器：全部 / 仅本地 / 仅远程 / 本地但不满足当前同步条件 / 已读完 / 未读完 / 公众号。
- 排序：最近阅读、进度、标题。
- 卡片操作：
  - 打开本地文件
  - 若本地不存在，可触发单本同步或提示需先同步
  - 显示过滤原因标签
- 卡片文案重点：
  - 封面
  - 书名 / 作者
  - 划线数
  - 进度百分比
  - 完成时间或最近阅读时间
  - 本地状态
  - 特殊标记（例如“公众号”“低于划线阈值”“黑名单”“未入白名单”）

7. 设置页入口的改法
- 在 `src/settingTab.ts` 增加一个“书架/画廊”设置区块。
- 这里不承担完整书架渲染，只提供：
  - 打开书架按钮
  - 可选的书架默认筛选配置（如果你后面想做）
- 这样满足“设置页面也可以增加入口”，同时保持设置页结构清晰。

8. 命令与入口改动点
- `main.ts` 里新增 command。
- ribbon 右键菜单里也建议新增“打开书架”入口，和现有“同步”“打开微信读书视图”保持一致。
- 如果采用独立 view，需要注册新的 view type，并封装激活逻辑，类似当前 `activateReadingView()`。

9. 打开本地文件能力
- 书架卡片应直接持有 `TFile` 或可解析到 `TFile` 的路径信息。
- 点击卡片主区域可打开详情/聚焦；点击“打开本地文件”按钮则直接在当前叶子页或新标签页中打开对应笔记。
- 对“本地存在但不满足当前同步条件”的书，这个能力尤其重要，因为它正是你这个需求的核心使用场景。

10. 与现有同步逻辑的边界
- 书架本身不应改写同步结果，也不要直接编辑用户文件。
- 它只是可视化当前“远程状态 + 本地状态 + 当前设置下的可同步性”。
- 过滤判断应尽量复用 `syncNotebooks.ts` 的规则来源，避免出现“书架显示能同步，但实际同步时被跳过”的分叉。

11. 推荐的实施顺序
- 先做数据模型与本地/远程合并。
- 再抽取“同步过滤原因”的判定函数。
- 然后做独立书架 view 的基础卡片列表。
- 再接入懒加载进度。
- 最后补设置入口、命令入口、打开本地文件、筛选与样式优化。

12. 这个方案在当前仓库里是可落地的
- 命令入口已有模式：`main.ts`
- 设置入口已有模式：`src/settingTab.ts`
- 本地文件索引已有基础：`src/fileManager.ts`
- 远程元数据已有基础：`parseMetadata()` + `ApiManager.getNotebooksWithRetry()`
- 单本进度接口已有基础：`ApiManager.getProgress()`
- 现有书籍卡片样式与选择器 UI 也能部分复用：`settingTab.ts` + `style.css`

如果你愿意，我下一步可以直接基于这个方案，给你拆成“最小可实现版本”的具体改动

清单。
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prompt>

Implement the plan:

这是一个需要代码改动的功能，但从你的描述看，更适合先按“功能设计/实施方案”来推进。

1. 先抽出“书架数据层”
- 在现有 `ApiManager`、`SyncNotebooks`、`FileManager` 之外新增一个专门的书架数据组装层。
- 统一合并两类数据：远程书架元数据（`/api/user/notebook`）和本地 vault 中已同步文件（frontmatter + 文件路径）。
- 为每本书生成书架卡片模型，至少包含：封面、书名、作者、bookId、划线数、评论数、本地文件路径、是否存在本地文件、远程最近阅读日期、进度加载状态、完成时间、阅读进度、是否公众号、是否命中过滤规则。

2. 明确“过滤状态”而不是只做简单布尔值
- 现有过滤逻辑分散在 `syncNotebooks.ts` 的 `filterNoteMetas` 中，书架功能需要把“为什么不会被同步”显式展示出来。
- 建议把过滤结果拆成多个原因维度，例如：
  - `excludedByArticleType`
  - `excludedByNoteCount`
  - `excludedByBlacklist`
  - `excludedByWhitelist`
  - `includedByCurrentSettings`
- 这样书架上就可以对“本地存在，但按当前设置不会参与同步”的书打特殊标记，而不是只显示“已过滤”。

3. 复用现有本地文件识别能力
- `fileManager.ts` 已经能扫描 vault 内所有微信读书笔记文件并识别 `bookId`。
- 书架应复用这套 frontmatter 识别逻辑，而不是重新扫描 Markdown 内容。
- 需要补一个“bookId -> 本地文件”索引，支持卡片上一键打开本地文件，并能区分：
  - 远程存在 + 本地存在

  - 仅远程存在
  - 仅本地存在（孤儿文件 / 当前远程已无此书）

4. 懒加载阅读进度
- 你提到的性能问题成立：`getProgress(bookId)` 目前是逐本请求，不能在书架首次打开时全量拉取。
- 书架列表首次只展示基础元数据；进入视口、翻页或用户点击“加载更多”时，再批量/分片加载当前页书籍的进度。
- 需要引入分页或游标式状态，至少支持：
  - 首屏卡片渲染
  - 当前页进度异步回填
  - 已加载结果缓存
  - 失败重试
- 如果用户切换筛选条件，只复用已有缓存，不重复请求已拿到的进度。

5. UI 形态建议用独立书架视图，而不是继续堆在设置页里
- 现有 `WereadReadingView` 说明插件已经有自定义 view 的模式。
- 书架更适合做成独立 `ItemView` 或专用 `Modal`，而不是把大量卡片塞进设置页。
- 设置页只保留“打开书架”的入口；命令面板和 ribbon 菜单也新增“打开微信读书书架”命令。
- 这样书架可以承载筛选、搜索、懒加载、打开本地文件等交互，而不会让设置页过重。

6. 书架界面建议支持的最小交互
- 搜索：按书名、作者过滤。
- 过滤器：全部 / 仅本地 / 仅远程 / 本地但不满足当前同步条件 / 已读完 / 未读完 / 公众号。
- 排序：最近阅读、进度、标题。
- 卡片操作：
  - 打开本地文件
  - 若本地不存在，可触发单本同步或提示需先同步
  - 显示过滤原因标签
- 卡片文案重点：
  - 封面
  - 书名 / 作者
  - 划线数
  - 进度百分比
  - 完成时间或最近阅读时间
  - 本地状态
  - 特殊标记（例如“公众号”“低于划线阈值”“黑名单”“未入白名单”）

7. 设置页入口的改法
- 在 `src/settingTab.ts` 增加一个“书架/画廊”设置区块。
- 这里不承担完整书架渲染，只提供：
  - 打开书架按钮
  - 可选的书架默认筛选配置（如果你后面想做）
- 这样满足“设置页面也可以增加入口”，同时保持设置页结构清晰。

8. 命令与入口改动点
- `main.ts` 里新增 command。
- ribbon 右键菜单里也建议新增“打开书架”入口，和现有“同步”“打开微信读书视图”保持一致。
- 如果采用独立 view，需要注册新的 view type，并封装激活逻辑，类似当前 `activateReadingView()`。

9. 打开本地文件能力
- 书架卡片应直接持有 `TFile` 或可解析到 `TFile` 的路径信息。
- 点击卡片主区域可打开详情/聚焦；点击“打开本地文件”按钮则直接在当前叶子页或新标签页中打开对应笔记。
- 对“本地存在但不满足当前同步条件”的书，这个能力尤其重要，因为它正是你这个需求的核心使用场景。

10. 与现有同步逻辑的边界
- 书架本身不应改写同步结果，也不要直接编辑用户文件。
- 它只是可视化当前“远程状态 + 本地状态 + 当前设置下的可同步性”。
- 过滤判断应尽量复用 `syncNotebooks.ts` 的规则来源，避免出现“书架显示能同步，但实际同步时被跳过”的分叉。

11. 推荐的实施顺序
- 先做数据模型与本地/远程合并。
- 再抽取“同步过滤原因”的判定函数。
- 然后做独立书架 view 的基础卡片列表。
- 再接入懒加载进度。
- 最后补设置入口、命令入口、打开本地文件、筛选与样式优化。

12. 这个方案在当前仓库里是可落地的
- 命令入口已有模式：`main.ts`
- 设置入口已有模式：`src/settingTab.ts`
- 本地文件索引已有基础：`src/fileManager.ts`
- 远程元数据已有基础：`parseMetadata()` + `ApiManager.getNotebooksWithRetry()`
- 单本进度接口已有基础：`ApiManager.getProgress()`
- 现有书籍卡片样式与选择器 UI 也能部分复用：`settingTab.ts` + `style.css`

如果你愿意，我下一步可以直接基于这个方案，给你拆成“最小可实现版本”的具体改动清单。